

Redundancy-Free Maximal Clique Enumeration via Search-Tree Reordering

Abstract

We study exact maximal clique enumeration in an undirected graph. A naive search over all vertex subsets is infeasible, and Bron-Kerbosch-style search greatly improves on this by expanding only clique-consistent candidates. However, even after a maximal clique C has been reported, the search still contains branches rooted at proper subsets of C . Those branches are known not to terminate inside C , yet standard depth-first search and pivoting do not use that information to reorganize the remaining search space.

We present REORDER_{SIB}, an exact algorithm based on recursive search-tree reordering. The central observation is that once a branch state M is known to lie inside a discovered maximal clique C , every undiscovered maximal clique extending M must escape C through a vertex in $Q \setminus C$. For a single covering clique, this yields an explicit reordering rule: outside vertices must be explored before inside vertices. For several covering cliques, the admissible escape choices become exactly the minimal clique-compatible hitting sets of the induced constraint family. These sets seed a replacement family of branches that preserves completeness while avoiding redundant traversal of subsets of known cliques.

We formalize this idea, prove soundness, completeness, and duplicate-freedom for the resulting core algorithm, and then describe how the implementation realizes the same principle using clique indexing, branch merging, and exact hitting-set solvers.

PVLDB Reference Format:

. Redundancy-Free Maximal Clique Enumeration via Search-Tree Reordering. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 Introduction

A *maximal clique* in an undirected graph $G = (V, E)$ is a complete subgraph that cannot be extended by any additional vertex. The problem of enumerating all maximal cliques is a fundamental primitive in graph mining, with applications in community detection [5], protein-complex identification [1], and network security analysis.

A direct way to solve the problem is to examine subsets of vertices and test whether each subset is a maximal clique. This view makes the combinatorial difficulty clear, but it is also hopeless in practice because there are $2^{|V|}$ subsets. Exact algorithms therefore organize the search much more carefully. The Bron-Kerbosch

algorithm [2] and its pivoting variant [6] provide the standard framework: they maintain a growing clique R , a set P of candidate extensions, and a set X of already processed vertices, and branch only on vertices that remain clique-consistent with the current partial solution. Combined with the degeneracy ordering of Eppstein, Löffler, and Strash [3], this framework achieves the worst-case optimal bound $O(\delta \cdot n \cdot 3^{\delta/3})$, where δ is the degeneracy of G .

A structural redundancy nevertheless remains. Suppose the search reports a maximal clique $C = \{a, b, c, d\}$. At that moment the search tree may still contain branches rooted at $\{a\}$, $\{a, b\}$, and $\{a, b, c\}$, because those branches were created before C was known. None of these states can terminate inside C : any new maximal clique reachable from $\{a, b, c\}$ must contain a vertex outside C , or else it would be a subset of C and therefore not maximal. Standard pivoting can delay some branches, but it does not systematically re-route the search once such information becomes available.

This paper is about exploiting that missing information. Our goal is not to replace clique-consistent search, but to refine it: after a clique is found, we want the remaining search to focus first on the vertices that can lead to genuinely new maximal cliques and to postpone vertices that are already explained by the discovered clique.

Our contribution. We present REORDER_{SIB}, an exact maximal-clique enumeration algorithm based on recursive search-tree reordering. Its central mechanism is simple: once a discovered maximal clique C covers a branch state M , the search below M must leave C , so vertices from $Q \setminus C$ are promoted to the front of the search and vertices from $Q \cap C$ are pushed to the tail. For one covering clique this gives an explicit reordering rule; for several covering cliques it leads to a minimal clique-compatible hitting-set problem whose solutions define a replacement family of branches.

The paper develops this idea in three stages. First, we formalize the structural facts that make reordering correct: canonical ownership of a clique by a unique root branch, the escape property induced by a discovered clique, and the sibling-set characterization of valid replacement branches. Second, we turn these facts into a clean core algorithm and prove soundness (Theorem 4.1), completeness (Theorem 4.2), and duplicate-freedom (Theorem 4.3). Third, we explain how the actual implementation realizes the same idea using clique indexing, branch merging, and exact hitting-set solvers.

The paper is organised as follows. Section 2 fixes notation. Section 3 turns the redundancy observed above into precise structural lemmas. Section 4 presents the core algorithm and its correctness proof. Section 5 then explains how the REORDER_{SIB} implementation realizes the same search principle in code.

2 Preliminaries

Let $G = (V, E)$ be a simple, undirected graph with $n = |V|$ vertices and $m = |E|$ edges. For a vertex $v \in V$, write $N(v)$ for its open neighborhood. For a set $S \subseteq V$, write $N(S) = (\bigcap_{v \in S} N(v)) \setminus S$ for

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

the set of vertices adjacent to every member of S but not in S itself. A *clique* is a set $K \subseteq V$ in which every pair of vertices is adjacent. A clique is *maximal* if no vertex outside it is adjacent to all of its members. We write $\mathcal{M}(G)$ for the family of all maximal cliques; the Moon–Moser bound gives $|\mathcal{M}(G)| \leq 3^{n/3}$ [4].

Vertex ordering. Fix a total order $\sigma: V \rightarrow [n]$ – in practice, the degeneracy order [3], computed in $O(n+m)$ by iteratively removing a minimum-degree vertex. Define the *forward neighborhood* of v as

$$N^+(v) = \{u \in N(v) : \sigma(u) > \sigma(v)\}.$$

Under the degeneracy order, $|N^+(v)| \leq \delta$ for every v , where δ is the degeneracy of G .

Branches. The search state at any point in the algorithm is captured by a *branch*.

Definition 2.1 (Branch). A *branch* is a triple (M, h, Q) where M is a clique in G , $h \in M$ is the *branch head*, and $Q \subseteq N(M) \setminus M$ is the *candidate set*. We call M the *must-in set*. The head h is an administrative label attached to the branch: root branches have $M = \{h\}$, and sibling-generated branches use as head the first post-sibling vertex when such a vertex exists. The head records the distinguished vertex that generated the current ordered subproblem; it is not part of the clique predicate itself. The *coverage* of the branch is

$$\text{Ext}(M, Q) = \{K \in \mathcal{M}(G) : M \subseteq K \text{ and } K \setminus M \subseteq Q\}.$$

A collection of branches $\mathcal{F}$ is *complete* for $S \subseteq \mathcal{M}(G)$ if $S \subseteq \bigcup_{(M_i, h_i, Q_i) \in \mathcal{F}} \text{Ext}(M_i, Q_i)$.

Intuitively, $\text{Ext}(M, Q)$ is the set of maximal cliques that this branch is responsible for finding: those that contain every vertex of M and draw their remaining members from Q . The goal of the algorithm is to maintain a complete forest $\mathcal{F}$ at every step while progressively shrinking it by finding cliques and pruning the branches that covered them. With this notation in place, we can now formalize the redundancy from the introduction and show how it leads to a principled reordering rule.

3 From Redundancy to Reordering

We now turn the informal redundancy from Section 1 into a sequence of structural statements. The progression is as follows. We first fix how a clique is reached inside an ordered branch. We then show that once a maximal clique has been discovered, any future clique extending one of its proper subsets must escape it. This escape property yields the reordering rule for one covering clique and, in the general case, the sibling-set replacement used by REORDERSIB.

3.1 Ordered DFS Within a Branch

The core search routine for a branch is an ordered DFS. If $K \in \text{Ext}(M, Q)$ and

$$K \setminus M = \{w_1, \dots, w_t\} \quad \text{with} \quad \sigma(w_1) < \dots < \sigma(w_t),$$

then $(w_1, \dots, w_t)$ is the *canonical extension sequence* of K relative to (M, Q) .

LEMMA 3.1 (CANONICAL REACHABILITY). *Let (M, h, Q) be a branch and let $K \in \text{Ext}(M, Q)$. If $K \setminus M = \{w_1, \dots, w_t\}$ is written in increasing σ -order, then ordered DFS can reach K by choosing $w_1, \dots, w_t$ in*

that order and recursing with candidate sets

$$Q_j = Q \cap N^+(w_1) \cap \dots \cap N^+(w_j), \quad 1 \leq j \leq t.$$

PROOF. Because $K \setminus M \subseteq Q$, the first vertex w_1 is available in Q . Since K is a clique, every w_j with $j \geq 2$ is adjacent to w_1 , and since the sequence is increasing in σ , each such w_j lies in $N^+(w_1)$. Hence $\{w_2, \dots, w_t\} \subseteq Q_1$.

Applying the same argument inductively, after choosing the prefix $w_1, \dots, w_j$, every remaining vertex $w_{j+1}, \dots, w_t$ is adjacent to all previously chosen vertices and has larger σ -rank than w_j , so it belongs to Q_j . Therefore the ordered DFS can follow the canonical extension sequence all the way to the state (K, Q_t) . We claim that $Q_t = \emptyset$. Indeed, if some $u \in Q_t$ existed, then $u \in Q \subseteq N(M)$, so u is adjacent to every vertex of M . Also, for each j , membership in $N^+(w_j)$ implies that u is adjacent to w_j . Thus u is adjacent to every vertex of $K = M \cup \{w_1, \dots, w_t\}$, contradicting the maximality of K . Therefore $Q_t = \emptyset$, so ordered DFS reaches K as a leaf. $\square$

3.2 Unique Root Ownership

Fix the total order σ from Section 2. The initial search starts from one root branch per vertex.

Definition 3.2 (Initial forest). The *initial forest* is

$$\mathcal{F}^0 = \{(\{v\}, v, N^+(v)) : v \in V\}.$$

Branch $(\{v\}, v, N^+(v))$ is the unique root branch of vertex v .

LEMMA 3.3 (UNIQUE ROOT OWNERSHIP). *Every maximal clique $K \in \mathcal{M}(G)$ belongs to exactly one root branch of $\mathcal{F}^0$, namely the branch seeded at its minimum-rank vertex*

$$v_* = \arg \min_{u \in K} \sigma(u).$$

PROOF. Let v_* be the minimum-rank vertex of K . Every vertex $u \in K \setminus \{v_*\}$ is adjacent to v_* and satisfies $\sigma(u) > \sigma(v_*)$, so $K \setminus \{v_*\} \subseteq N^+(v_*)$. Hence $K \in \text{Ext}(\{v_*\}, N^+(v_*))$.

Conversely, suppose $K \in \text{Ext}(\{u\}, N^+(u))$ for some $u \neq v_*$. Then every other vertex of K has rank strictly greater than $\sigma(u)$, which implies $\sigma(v_*) > \sigma(u)$, contradicting the definition of v_* . Thus K belongs to exactly one root branch. $\square$

Lemma 3.3 is the source of duplicate-freedom at the root level: every maximal clique has exactly one initial owner.

3.3 Escape From a Known Clique

Suppose a branch (M, h, Q) is being processed and a maximal clique $C \supseteq M$ has already been found. The next lemma captures the structural fact that drives reordering.

LEMMA 3.4 (ESCAPE LEMMA). *Let (M, h, Q) be a branch and let $\mathcal{K}$ be the set of maximal cliques found so far. For any $K \in \text{Ext}(M, Q) \setminus \mathcal{K}$ and any covering clique*

$$C \in \mathcal{C}(M, \mathcal{K}) := \{C' \in \mathcal{K} : M \subseteq C'\},$$

we have

$$K \cap (Q \setminus C) \neq \emptyset.$$

PROOF. Since $K \notin \mathcal{K}$, we have $K \neq C$. Because both K and C are maximal cliques, neither can properly contain the other. Since both contain M , they must differ outside M . Thus $K \setminus C \neq \emptyset$. For any

vertex $v \in K \setminus C$, we also have $v \in K \setminus M \subseteq Q$ by the definition of $\text{Ext}(M, Q)$. Hence $v \in Q \setminus C$, proving the claim. $\square$

Lemma 3.4 says that once $M \subseteq C$ is known, the search can never finish inside C again. Every future maximal clique extending M must *escape* C .

3.4 Re-ordering After One Covering Clique

The escape lemma immediately yields a re-ordering principle in the case of a single covering clique.

LEMMA 3.5 (SINGLE-CLIQUE RE-ORDERING). *Let (M, h, Q) be a branch and let C be a discovered maximal clique with $M \subseteq C$. Write*

$$Q_{\text{out}} := Q \setminus C, \quad Q_{\text{in}} := Q \cap C.$$

Then every clique $K \in \text{Ext}(M, Q) \setminus \mathcal{K}$ can be written as

$$K = M \cup S \cup T, \quad \emptyset \neq S \subseteq Q_{\text{out}}, \quad T \subseteq Q_{\text{in}},$$

where $M \cup S \cup T$ is a clique. Consequently, any exhaustive search may postpone all vertices of Q_{in} until after it has selected at least one vertex from Q_{out} .

PROOF. Take any $K \in \text{Ext}(M, Q) \setminus \mathcal{K}$. By Lemma 3.4, K contains at least one vertex in $Q \setminus C$. Let $S := K \cap (Q \setminus C)$ and $T := K \cap (Q \cap C)$. Then $S \neq \emptyset$, $S \subseteq Q_{\text{out}}$, and $T \subseteq Q_{\text{in}}$. Since K is a clique and $K \setminus M \subseteq Q$, we obtain $K = M \cup S \cup T$ with the stated properties. The search-order conclusion follows immediately: every valid continuation must pick from Q_{out} before any clique completed only inside Q_{in} could be relevant. $\square$

This is the formal version of the central design principle: once a clique is known, its reusable vertices are pushed to the tail of the search, while the outside vertices that enable genuinely new maximal cliques are moved to the front.

In the motivating example, once clique $\{a, b, c\}$ is known, any later maximal clique extending $\{a, c\}$ must first use some outside vertex such as d . Re-ordering therefore means searching through d before revisiting the already-explained vertices of $\{a, b, c\}$.

3.5 Sibling Sets Under Multiple Covering Cliques

If several previously found maximal cliques cover the same branch, a new clique must escape *all* of them simultaneously. This produces a natural hitting-set problem.

Definition 3.6 (Sibling set). For a branch (M, h, Q) and discovered-clique set $\mathcal{K}$, define the constraint family

$$\mathcal{H}(M, Q, \mathcal{K}) = \{Q \setminus C : C \in \mathcal{C}(M, \mathcal{K})\}.$$

A set $S \subseteq Q$ is a *sibling set* if it satisfies:

- (1) *Hitting*: $S \cap H \neq \emptyset$ for every $H \in \mathcal{H}(M, Q, \mathcal{K})$.
- (2) *Clique compatibility*: S is a clique in G .
- (3) *Minimality*: no proper subset of S satisfies both (1) and (2).

We write $\mathcal{S}(M, Q, \mathcal{K})$ for the family of all sibling sets.

For a sibling set S , the remaining common expansion vertices are

$$\text{CE}(Q, S) = (Q \setminus S) \cap \bigcap_{v \in S} N(v).$$

LEMMA 3.7 (SIBLING REPLACEMENT IS COMPLETE). *Let (M, h, Q) be a branch and let $\mathcal{K}$ be the set of already discovered maximal cliques. Define*

$$\mathcal{F}_0^S = \{(M \cup S, h, \emptyset) : S \in \mathcal{S}(M, Q, \mathcal{K}), \text{CE}(Q, S) = \emptyset\},$$

$$\mathcal{F}_1^S = \left\{ (M \cup S \cup \{v\}, v, \text{CE}(Q, S) \cap N^+(v)) : \begin{smallmatrix} S \in \mathcal{S}(M, Q, \mathcal{K}), \\ v \in \text{CE}(Q, S) \end{smallmatrix} \right\},$$

and let $\mathcal{F}^S = \mathcal{F}_0^S \cup \mathcal{F}_1^S$. Then $\mathcal{F}^S$ is complete for $\text{Ext}(M, Q) \setminus \mathcal{K}$.

PROOF. Let $K \in \text{Ext}(M, Q) \setminus \mathcal{K}$. By Lemma 3.4, $K \setminus M$ hits every set in $\mathcal{H}(M, Q, \mathcal{K})$. Since K is a clique, $K \setminus M$ is also clique-compatible. Therefore $K \setminus M$ contains some inclusion-minimal clique-compatible hitting subset S_K , and by construction $S_K \in \mathcal{S}(M, Q, \mathcal{K})$.

If $K = M \cup S_K$, then $\text{CE}(Q, S_K) = \emptyset$. Otherwise some vertex $u \in \text{CE}(Q, S_K)$ would lie in $Q \setminus S_K$, would be adjacent to every vertex of S_K by definition of CE, and would be adjacent to every vertex of M because $Q \subseteq N(M)$. Hence u would be adjacent to every vertex of the clique $K = M \cup S_K$, contradicting the maximality of K . Therefore $K \in \text{Ext}(M \cup S_K, \emptyset)$, which is represented by the leaf branch in $\mathcal{F}^S$.

Otherwise let w be the minimum-rank vertex of $K \setminus (M \cup S_K)$. Every vertex in $K \setminus (M \cup S_K \cup \{w\})$ lies in $\text{CE}(Q, S_K)$ because it belongs to $Q \setminus S_K$ and is adjacent to every vertex of S_K . By minimality of w , every such vertex also lies in $N^+(w)$. Hence

$$K \in \text{Ext}(M \cup S_K \cup \{w\}, \text{CE}(Q, S_K) \cap N^+(w)),$$

and the corresponding branch is present in $\mathcal{F}^S$. $\square$

Lemma 3.7 is the formal sibling effect: instead of traversing all descendants of a non-maximal subset M , we replace that search space by the smaller family of branches generated from minimal escape sets.

COROLLARY 3.8 (NO SIBLING SET, NO UNDISCOVERED CLIQUE). *Let (M, h, Q) be a branch. If $\mathcal{S}(M, Q, \mathcal{K}) = \emptyset$, then $\text{Ext}(M, Q) \setminus \mathcal{K} = \emptyset$.*

PROOF. If some $K \in \text{Ext}(M, Q) \setminus \mathcal{K}$ existed, then the proof of Lemma 3.7 would produce an inclusion-minimal clique-compatible hitting subset $S_K \in \mathcal{S}(M, Q, \mathcal{K})$. This contradicts the assumption that $\mathcal{S}(M, Q, \mathcal{K}) = \emptyset$. $\square$

4 The Core REORDERSIB Algorithm

4.1 Core Search Procedure

The previous section identified the mathematical structure behind the reordering idea. We now package those facts into a clean core algorithm. The algorithm maintains a global set $\mathcal{K}$ of discovered maximal cliques and a guard set $\mathcal{D}$ of already reported empty-candidate branches. It starts from the initial forest $\mathcal{F}^0$ and repeatedly alternates between two actions.

For clarity, this section presents a forest-level core algorithm: a discovered clique reorders the residual forest in one step. Section 5 then explains how the code realizes the same transformations through a more local branch-by-branch recursion.

Sibling phase. Before a non-root branch is expanded, all already found maximal cliques covering its must-in set are collected. The branch is then replaced by the sibling-generated family from Lemma 3.7. If several generated branches have the same must-in set, they are merged by taking the union of their candidate sets; the

Algorithm 1 RCALL: sibling reduction and enumeration dispatch

Require: Forest $\mathcal{F} = \{(M_i, h_i, Q_i)\}$, level $\ell \geq 0$, global discovered set $\mathcal{K}$, guard set $\mathcal{D}$

```
1: if  $\ell > 0$  and  $\mathcal{F} \neq \emptyset$  then
2:    $(M, h, Q) \leftarrow (M_1, h_1, Q_1)$ 
3:    $\mathcal{F}_{\text{rest}} \leftarrow \mathcal{F} \setminus \{(M, h, Q)\}$ 
4:    $\mathcal{S} \leftarrow \mathcal{S}(M, Q, \mathcal{K})$ 
5:   if  $\mathcal{S} = \emptyset$  then
6:      $\mathcal{F} \leftarrow \mathcal{F}_{\text{rest}}$ 
7:   else
8:      $\mathcal{F}_{\text{new}} \leftarrow \emptyset$ 
9:     for all  $S \in \mathcal{S}$  do
10:       $Q_S \leftarrow \text{CE}(Q, S)$ 
11:      if  $Q_S = \emptyset$  then
12:         $\mathcal{F}_{\text{new}} \leftarrow \mathcal{F}_{\text{new}} \cup \{(M \cup S, h, \emptyset)\}$ 
13:      else
14:        for all  $v \in Q_S$  do
15:           $\mathcal{F}_{\text{new}} \leftarrow \mathcal{F}_{\text{new}} \cup \{(M \cup S \cup \{v\}, v, Q_S \cap N^+(v))\}$ 
16:        end for
17:      end if
18:    end for
19:     $\mathcal{F} \leftarrow \mathcal{F}_{\text{new}} \cup \mathcal{F}_{\text{rest}}$ 
20:  end if
21:  Merge branches with identical must-in sets by unioning
  their candidate sets and retaining an arbitrary head label
22: end if
23: for all  $(M_i, h_i, Q_i) \in \mathcal{F}$  do
24:    $\text{done} \leftarrow \text{false}$ 
25:    $\text{ENUMERATE}(M_i, h_i, Q_i, \mathcal{F}, i, \ell, \text{done})$ 
26:   if  $\text{done}$  then break
27: end if
28: end for
```

stored head label may then be chosen arbitrarily, since coverage depends only on the pair (M, Q) .

Enumeration phase. DFS is run on the current branches in order. The DFS branches in increasing σ -order, always recursing with $Q \cap N^+(v)$ so that the next expansion is ordered by the new head vertex v . As soon as one maximal clique C is found, the current DFS stops, C is added to $\mathcal{K}$, and the residual forest is rebuilt from the current branch together with all later branches by enlarging each candidate set with the vertices of C that are compatible with its must-in set. A leaf R is reported only if it is globally maximal, i.e., $N(R) = \emptyset$. The algorithm then recurses on the resulting residual forest, where the next sibling phase can use C as a covering clique.

This is the recursive re-ordering strategy: a discovered clique is used immediately to move already-explained vertices to the tail and to expose the outside vertices that can still produce new maximal cliques.

4.2 Correctness

We now prove soundness, completeness, and duplicate-freeness for the core algorithm.

Algorithm 2 ENUMERATE: ordered DFS with post-discovery re-ordering

```
1: procedure  $\text{ENUMERATE}(R, h, Q, \mathcal{F}, i, \ell, \text{done})$ 
2:   if  $Q = \emptyset$  then
3:     if  $N(R) = \emptyset$  and  $R \notin \mathcal{D}$  then
4:        $\mathcal{D} \leftarrow \mathcal{D} \cup \{R\}$ 
5:        $\mathcal{K} \leftarrow \mathcal{K} \cup \{R\}; \text{ done} \leftarrow \text{true}$ 
6:        $\mathcal{F}' \leftarrow \emptyset$ 
7:       for all  $(M_j, h_j, Q_j) \in \mathcal{F}$  with  $j \geq i$  do
8:          $Q'_j \leftarrow (Q_j \cup (R \cap N(M_j))) \setminus M_j$ 
9:          $\mathcal{F}' \leftarrow \mathcal{F}' \cup \{(M_j, h_j, Q'_j)\}$ 
10:      end for
11:       $\text{RCALL}(\mathcal{F}', \ell + 1)$ 
12:    end if
13:    return
14:  end if
15:  for all  $v \in Q$  in increasing  $\sigma$ -order do
16:     $\text{ENUMERATE}(R \cup \{v\}, v, Q \cap N^+(v), \mathcal{F}, i, \ell, \text{done})$ 
17:    if  $\text{done}$  then return
18:  end if
19: end for
20: end procedure
```

THEOREM 4.1 (SOUNDNESS). *Every set reported by REORDERSIB is a maximal clique of G .*

PROOF. The algorithm reports a set R only at a leaf with $Q = \emptyset$ and $N(R) = \emptyset$. Since every recursive extension adds only vertices adjacent to all previously chosen vertices, the reported set R is a clique. The condition $N(R) = \emptyset$ is exactly the definition of global maximality. Hence every reported set is a maximal clique. $\square$

THEOREM 4.2 (COMPLETENESS). *REORDERSIB reports every maximal clique in $\mathcal{M}(G)$.*

PROOF. We maintain the invariant that at the start of every call $\text{RCALL}(\mathcal{F}, \ell)$, the forest $\mathcal{F}$ is complete for $\mathcal{M}(G) \setminus \mathcal{K}$.

Base case. Initially, $\mathcal{K} = \emptyset$ and $\mathcal{F} = \mathcal{F}^0$. By Lemma 3.3, $\mathcal{F}^0$ is complete for $\mathcal{M}(G)$.

Sibling phase. When $\ell > 0$, the first branch (M, h, Q) is replaced by the family from Lemma 3.7. If $\mathcal{S}(M, Q, \mathcal{K}) = \emptyset$, then Corollary 3.8 shows that no undiscovered maximal clique remains in $\text{Ext}(M, Q)$, so the branch may be dropped. Otherwise the replacement family is complete for $\text{Ext}(M, Q) \setminus \mathcal{K}$, while every other branch is left unchanged. Hence completeness of the whole forest is preserved.

Existence of the next report. Since $\mathcal{F}$ is complete for $\mathcal{M}(G) \setminus \mathcal{K}$, if this set is non-empty then some branch (M, h, Q) covers a maximal clique $K \in \mathcal{M}(G) \setminus \mathcal{K}$. By Lemma 3.1, ordered DFS on that branch can reach K along its canonical extension sequence and reaches it as a leaf. Since K is a maximal clique, the reporting test $N(K) = \emptyset$ is satisfied at that leaf. Because the DFS is finite and exhaustive within each branch, the algorithm eventually reports some maximal clique $C \in \mathcal{M}(G) \setminus \mathcal{K}$.

Forest update after a report. Suppose DFS reports a maximal clique C . Let $\mathcal{K}' = \mathcal{K} \cup \{C\}$. Consider any $K \in \mathcal{M}(G) \setminus \mathcal{K}'$. Since branch i is the first branch in the current call that reports a clique, every earlier branch $1, \dots, i-1$ has already been searched exhaustively

and contains no clique in $\mathcal{M}(G) \setminus \mathcal{K}'$. It therefore suffices to consider branches $j \geq i$.

By the induction hypothesis, K was covered by some pre-update branch (M_j, h_j, Q_j) with $j \geq i$. In the residual forest, this branch is kept with updated candidate set

$$Q'_j = (Q_j \cup (C \cap N(M_j))) \setminus M_j.$$

Since $Q_j \subseteq Q'_j$, every clique that was covered by (M_j, h_j, Q_j) is still covered by (M_j, h_j, Q'_j) . In particular, $K \in \text{Ext}(M_j, Q'_j)$. Hence the residual forest remains complete for $\mathcal{M}(G) \setminus \mathcal{K}'$.

By induction, every maximal clique is eventually reported. $\square$

THEOREM 4.3 (DUPLICATE-FREEDOM). *REORDER_SIB reports each maximal clique at most once.*

PROOF. There are three possible sources of duplication.

Within one DFS branch. In ENUMERATE, once the DFS chooses a head vertex v , all later recursion is restricted to $N^+(v)$. Hence the vertices added after the head are visited in strictly increasing σ -order, so a fixed branch cannot report the same maximal clique twice.

Across sibling-generated branches with the same must-in set. The algorithm explicitly merges such branches before DFS begins, so they cannot lead to duplicate reports.

Across different later branches in the same forest. Suppose a maximal clique K could be reported again after it has already been discovered. This cannot happen because every reported clique is inserted into the guard set $\mathcal{D}$ immediately. Any later leaf equal to K is therefore discarded without being reported.

Hence no maximal clique is reported more than once. $\square$

5 Implementation Realization

The previous section described the theorem-level algorithm used for the correctness proof. We now return to the concrete implementation in `helpers.cpp`. The implementation realizes the same reordering idea, but with explicit branch-management data structures and more aggressive operational shortcuts.

Branch representation. The implementation stores each branch as a pair of arrays: `mustin[i]` for the must-in set M and `expandTo[i]` for the candidate set Q . The code does not store the abstract head h explicitly. For ordinary ordered branches, the controlling vertex is recovered from `mustin[i].back()`, while sibling-generated branches are tracked through `fullSkipCheck` and whole-branch containment tests rather than through a separate head field. The recursive driver `rCall` manages the current local branch family, while `enumerate` performs the DFS and triggers the next reordering step when it finds a clique.

Covering-clique retrieval. All discovered cliques are stored globally in `allCliques`. To avoid scanning that list naively, the code maintains a two-tier index

$$\mathcal{I}[v][\ell']$$

whose entries are clique IDs containing vertex v and discovered at level ℓ' . The function `collectCoveringCliques` chooses the least frequent vertex of the must-in set as a seed, filters on that vertex, and then verifies containment of the whole must-in set. This is the implementation counterpart of the set $\mathcal{C}(M, \mathcal{K})$.

Sibling effect. Given a branch (M, Q) and its covering cliques, the implementation constructs the constraint family $\{Q \setminus C\}$ and solves the corresponding clique-compatible hitting-set problem in `generateSiblingSetsFromCliques`. The function `commonExpand` computes the common expansion $\text{CE}(Q, S)$. The local forest created by `rCall` is exactly the branch family described in Lemma 3.7, except that the code materializes the family one local subproblem at a time and then merges any two generated branches with the same must-in set by unioning their candidate sets.

Ordered DFS and recursive reordering. Inside `enumerate`, recursion on a chosen vertex v uses `adjList2[v]`, which is the stored forward neighborhood $N^+(v)$ under the fixed vertex order. When a clique is found, the implementation stops the current DFS immediately, inserts the clique into the global discovered set, and rebuilds the residual search space before making the next recursive `rCall`. Unlike the forest-level presentation in Section 4, the code launches one recursive call per surviving reordered branch. This is an implementation choice, not a different search principle: known vertices are still pushed to the tail, and new escape vertices are still brought to the front.

Eager branch subsumption in the implementation. The clean core algorithm keeps the current branch and all later branches in the residual forest, then lets the next sibling phase use the newly reported clique as a covering clique. The implementation is more aggressive: it may delete a branch immediately when cheaper operational tests indicate that the remaining branch space has already been explained. Ordinary branches use the last mandatory vertex. Sibling-generated branches additionally use `fullSkipCheck` together with `branchSpaceInsideClique`. These are implementation-level accelerations of the same reordering idea.

Conceptual maximality guard. The core algorithm in Section 4 uses the explicit predicate $N(R) = \emptyset$ when a DFS leaf is reached. The implementation does not perform that test as a separate subroutine. Instead, it relies on the reordered frontier, branch deletion, and candidate pulling to ensure that non-maximal leaves are operationally suppressed before they would be reported. The explicit guard is part of the clean theorem-level specification.

Duplicate guards and aggressive branch management. The code keeps an explicit set `claimedEmptyBranches` to block repeated reports from empty-candidate branches, mirroring the guard set $\mathcal{D}$ in Algorithm 2. It also uses `fullSkipCheck` to mark branches produced by sibling replacement, allowing the implementation to decide whether a whole branch space is already contained in a newly found clique. These devices are part of the concrete implementation, not part of the abstract mathematical interface of a branch.

Exact hitting-set solvers. Two exact solvers are implemented.

Backtracking branch-and-bound. A DFS grows a current set $S_{\text{cur}} \subseteq Q$ in increasing order. As soon as S_{cur} hits all constraints and remains a clique, the branch stops: no proper superset can be minimal. A minimal-by-inclusion filter is applied to the resulting solutions.

Bitmask exact solver. When the number of constraints is at most 63, the coverage of a candidate vertex is stored as a bitmask. The DFS then uses constant-time coverage updates and three exact

reductions: unit propagation, constraint subsumption, and fail-first ordering of constraints by size. A fourth pruning rule skips any candidate that contributes no uncovered constraint, since such a candidate cannot participate in a minimal hitting set at that point. The solver maintains a live minimal-by-inclusion solution set, so no separate dominance pass is required after the search.

Pruning reductions. The implementation also applies additional structural reductions before the solver is invoked. Dominance pruning is exact: it removes any covering clique C_i that is a proper subset of another covering clique C_j , because the tighter constraint $Q \setminus C_j$ already implies $Q \setminus C_i$. The implementation also uses a level filter to limit covering-clique retrieval to recent recursive levels. This filter is an engineering device of the concrete implementation and is not part of the theorem-level core proof.

References

- [1] Gary D Bader and Christopher WV Hogue. 2003. An automated method for finding molecular complexes in large protein interaction networks. *BMC Bioinformatics* 4, 1 (2003), 1–27.
- [2] Coen Bron and Joep Kerbosch. 1973. Algorithm 457: finding all cliques of an undirected graph. *Commun. ACM* 16, 9 (1973), 575–577.
- [3] David Eppstein, Maarten Löffler, and Darren Strash. 2010. Listing all maximal cliques in sparse graphs in near-optimal time. In *International Symposium on Algorithms and Computation (ISAAC)*. Springer, 403–414.
- [4] John W Moon and Leo Moser. 1965. On cliques in graphs. *Israel Journal of Mathematics* 3, 1 (1965), 23–28.
- [5] Gergely Palla, Imre Derényi, Illés Farkas, and Tamás Vicsek. 2005. Uncovering the overlapping community structure of complex networks in nature and society. *Nature* 435, 7043 (2005), 814–818.
- [6] Etsuji Tomita, Akira Tanaka, and Haruhisa Takahashi. 2006. The worst-case time complexity for generating all maximal cliques and computational experiments. *Theoretical Computer Science* 363, 1 (2006), 28–42.